

โครงการรายวิชาการประมวลผลภาษาธรรมชาติ (Natural Language Processing)

รายงานการจำแนกข้อความภาษาไทยเพื่อระบุภาวะซึมเศร้า ด้วยเทคนิค Machine Learning และ Deep Learning

Thai Text Classification for Depression Detection: A Comparative Study

กลุ่มผู้จัดทำ:	Group 10 (กลุ่มที่ 10)
รายวิชา:	[ชื่อรายวิชา]
สมาชิกกลุ่ม:	[ใส่ภายหลัง]
อาจารย์ผู้สอน:	[ใส่ภายหลัง]
ภาคการศึกษา / ปี:	ภาคการศึกษาที่ 1 / 2026

เอกสารรายงานวิชาการสำหรับการนำเสนอและประเมินผลการเรียนรู้

1. ที่มาและความสำคัญ (Background & Significance)

ภาวะซึมเศร้า (Depression) เป็นปัญหาสุขภาพจิตที่ส่งผลกระทบต่อคุณภาพชีวิตของประชากรทั่วโลกอย่างมีนัยสำคัญ ในยุคดิจิทัล ผู้คนจำนวนมากเลือกที่จะระบายความรู้สึก อารมณ์ และประสบการณ์ส่วนตัวผ่านสื่อสังคมออนไลน์หรือแพลตฟอร์มสนทนา การประมวลผลภาษาธรรมชาติ (Natural Language Processing: NLP) และเทคนิคการจำแนกข้อความ (Text Classification) จึงมีบทบาทสำคัญในการนำมาใช้วิเคราะห์และคัดกรองข้อความภาษาไทยที่มีลักษณะบ่งชี้ถึงความเสี่ยงของภาวะซึมเศร้า

โครงการนี้เป็นการพัฒนาระบบจำแนกข้อความภาษาไทยแบบสองคลาส (Binary Text Classification) โดยมีนิยามของคลาสเป้าหมายดังนี้:

- Class 0 (ไม่ป่วย / Non-Depression):** ข้อความสนทนาทั่วไป ข้อความเชิงบวก หรือข้อความที่ไม่มีลักษณะบ่งชี้ถึงภาวะซึมเศร้า
- Class 1 (ป่วย / Depression):** ข้อความที่มีการบรรยายอารมณ์เศร้า ความสิ้นหวัง ความรู้สึกไร้ค่า อาการปิดกั้นทางจิตใจ หรือพฤติกรรมเสี่ยง

ข้อความร่วทางวิชาการ: ระบบที่พัฒนาขึ้นในโครงการนี้มีวัตถุประสงค์เพื่อการศึกษา ค้นคว้า และวิเคราะห์ทางเทคนิคด้าน NLP และ Text Classification เท่านั้น ห้ามนำผลลัพธ์ของแบบจำลองไปใช้ในการวินิจฉัยทางการแพทย์หรือตัดสินใจทางจิตเวชโดยตรง โดยระบบทำหน้าที่เป็นเพียงเครื่องมือวิเคราะห์เชิงสถิติและคัดกรองเบื้องต้น

2. วัตถุประสงค์ (Objectives)

โครงการนี้มีวัตถุประสงค์หลักในการดำเนินงานดังต่อไปนี้:

- เพื่อพัฒนาและประเมินประสิทธิภาพของแบบจำลองการจำแนกข้อความภาษาไทย (Thai Text Classification) สำหรับระบุข้อความที่มีลักษณะบ่งชี้ภาวะซึมเศร้า
- เพื่อศึกษาและเปรียบเทียบเทคนิคการแปลงข้อความเป็นฟีเจอร์ (Feature Extraction / Representation) ทั้งรูปแบบ Bag-of-Words (BoW), TF-IDF และ Pre-trained Word2Vec Embedding
- เพื่อศึกษาและออกแบบสถาปัตยกรรมโครงข่ายประสาทเทียมแบบ Deep Sequential Model ด้วย Bidirectional Long Short-Term Memory (BiLSTM) ร่วมกับการตัดคำภาษาไทย
- เพื่อเปรียบเทียบประสิทธิภาพของแบบจำลองการเรียนรู้ของเครื่องแบบดั้งเดิม (Traditional Machine Learning) ได้แก่ Logistic Regression และ Decision Tree ร่วมกับการปรับจูน Hyperparameter
- เพื่อประเมินประสิทธิภาพของแบบจำลองผ่านตัวชี้วัดมาตรฐาน (Accuracy, Precision, Recall, F1-score, ROC-AUC) และวิเคราะห์ความเหมาะสมในการประยุกต์ใช้งาน
- เพื่อเตรียมโครงสร้างและแนวทางสำหรับการขยายผลไปสู่แบบจำลอง Transformer-based Language Model (BERT) ในขั้นตอนต่อไป

3. ข้อมูลชุดข้อมูล (Dataset)

ข้อมูลที่ใช้ในการทดลองคือชุดข้อมูลข้อความภาษาไทยจากไฟล์ Depression_Dataset.csv โดยข้อมูลทั้งหมดถูกตรวจสอบและสกัดค่าทางสถิติจริงจาก Jupyter Notebook ของ Group 10 ดังแสดงในตารางที่ 1

รายการ (Attribute)	จำนวน / ค่าทางสถิติที่พบใน Notebook
จำนวนข้อมูลทั้งหมด (Total Dataset)	32,676 แถว
จำนวนข้อมูล Class 0 (ไม่ป่วย / Non-Depression)	16,138 แถว (49.39%)
จำนวนข้อมูล Class 1 (ป่วย / Depression)	16,538 แถว (50.61%)
ค่าสูญหาย (Missing Values ในคอลัมน์ text)	0 แถว (ไม่พบค่าว่าง)
จำนวนแถวซ้ำซ้อนใน Raw Data (Duplicate Rows)	5,096 แถว
จำนวนแถวซ้ำซ้อนหลังทำความสะอาด (Duplicate Clean Texts)	5,096 แถว
ความยาวข้อความต่ำสุด (Min Character Length)	3 ตัวอักษร
ความยาวข้อความสูงสุด (Max Character Length)	4,496 ตัวอักษร
ความยาวข้อความเฉลี่ย (Mean Character Length)	91.95 ตัวอักษร (มัธยฐาน = 61.0 ตัวอักษร)

หมายเหตุ: สัดส่วนของข้อมูลระหว่าง Class 0 (49.39%) และ Class 1 (50.61%) มีความสมดุลสูง (Balanced Class Distribution) จึงไม่ก่อให้เกิดปัญหา Class Imbalance ชั้รุนแรง

4. การเตรียมข้อมูลก่อนการประมวลผล (Data Preprocessing)

กระบวนการเตรียมข้อมูลตามที่มีการดำเนินการจริงใน Notebook ประกอบด้วยขั้นตอนที่เป็นระบบดังต่อไปนี้:

- การตรวจสอบ Missing Values:** ตรวจสอบค่าว่างในคอลัมน์ text พบว่าไม่มีค่าว่าง (Empty Text = 0)
- การตรวจสอบและจัดการข้อมูลซ้ำซ้อน (Duplicate Handling):** ตรวจสอบแถวที่ซ้ำซ้อนในชุดข้อมูลเพื่อวิเคราะห์ความขัดแย้งของ Label
- การทำความสะอาดข้อความ (Text Cleaning):**
 - ลบ URL / Hyperlinks ด้วย Regular Expression: `re.sub(r'https?:\/\/\S+|www\.\S+', '', text)`
 - ลบช่องว่างส่วนเกิน (Excessive Whitespace): `re.sub(r'\s+', ' ', text)`
 - ตัดช่องว่างหัวและท้ายข้อความด้วย `text.strip()`
- การตัดคำภาษาไทย (Thai Tokenization):** ใช้ไลบรารี `pythainlp.tokenize.word_tokenize` ด้วย engine `newmm` (Maximal Matching) สำหรับแยกคำในประโยคภาษาไทย
- การแบ่งชุดข้อมูล (Dataset Splitting):** แบ่งข้อมูลออกเป็น 3 ส่วนในสัดส่วน **70% : 15% : 15%** แบบคงสัดส่วนคลาส

Raw Dataset (32,676 แถว)
↓
Data Cleaning (ลบ URL, จัดการ Whitespace, Strip)
↓
Duplicate & Label Verification (ตรวจสอบความถูกต้องของข้อความ)
↓
Thai Tokenization (PyThaiNLP: newmm)
↓
Train / Validation / Test Split (สัดส่วน 70% / 15% / 15%)
↓
Feature Extraction (Word2Vec / Bag-of-Words / TF-IDF)
↓
Model Training (BiLSTM / Logistic Regression / Decision Tree)
↓
Evaluation on Test Set (Accuracy, Precision, Recall, F1-score, ROC-AUC)

ชุดข้อมูล (Dataset Split)	จำนวนแถว	Class 0 (ปกติ)	Class 1 (ซึมเศร้า)	สัดส่วน (%)
Train Set	22,873	11,297 (49.39%)	11,576 (50.61%)	70.0%
Validation Set	4,901	2,420 (49.38%)	2,481 (50.62%)	15.0%
Test Set	4,902	2,421 (49.39%)	2,481 (50.61%)	15.0%
รวมทั้งหมด (Total)	32,676	16,138 (49.39%)	16,538 (50.61%)	100.0%

5. สภาพแวดล้อมและการตั้งค่าการทดลอง (Experimental Setup)

การทดลองทั้งหมดในโครงการดำเนินการภายใต้สภาพแวดล้อมซอฟต์แวร์และไลบรารีมาตรฐานที่ระบุใน Notebook ดังนี้:

- **ภาษาและแพลตฟอร์ม:** Python (ผ่าน Jupyter Notebook / Google Colab Environment)
- **การจัดการข้อมูลและการคำนวณ:** pandas, numpy
- **การประมวลผลภาษาไทย:** pythainlp (Tokenization engine: newmm)
- **การสร้าง Word Embedding:** gensim.models.Word2Vec
- **Machine Learning:** scikit-learn (Logistic Regression, Decision Tree, CountVectorizer, TfidfVectorizer, Metrics)
- **Deep Learning:** tensorflow / keras (Sequential, Embedding, Bidirectional, LSTM, Dense, Dropout)
- **การสร้างภาพกราฟิก:** matplotlib, seaborn
- **Random State:** มีการกำหนด seed = 42 ใน Word2Vec และ random_state = 42 ใน Decision Tree เพื่อให้ผลการทดลองสามารถทำซ้ำได้ (Reproducibility)
- **การประมวลผลฮาร์ดแวร์:** รันบน CPU / Standard Execution Environment (พบข้อความ TensorFlow แจ้งเตือน native Windows CPU fallback)

6. Model 1: Word2Vec Embedding + BiLSTM (Deep Sequential Model)

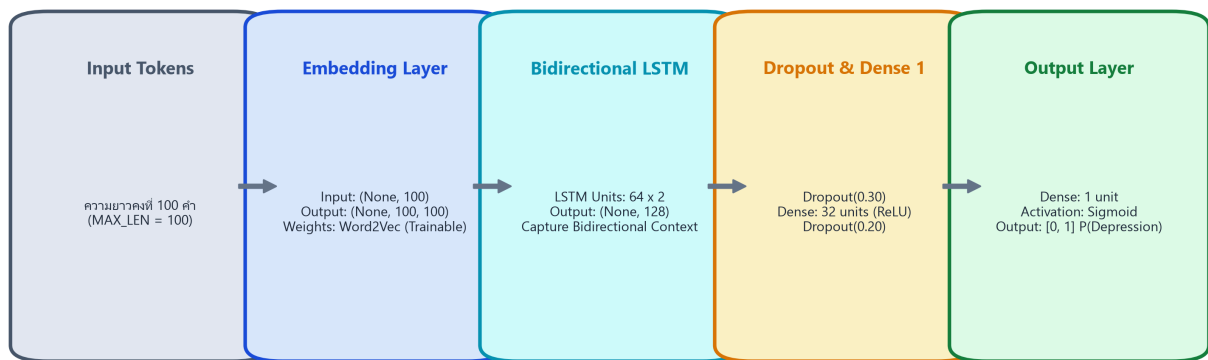
แนวคิดและหลักการ: แบบจำลองโครงข่ายประสาทเทียมแบบลำดับเวลา (Sequential Neural Network) ที่ใช้เทคนิค **Word2Vec (Skip-gram)** ในการแปลงคำศัพท์เป็นเวกเตอร์ความหมายหนาแน่น (Dense Vector) ขนาด 100 มิติ จากนั้นส่งต่อไปยังโครงข่าย **Bidirectional LSTM (BiLSTM)** ซึ่งมีความสามารถในการประมวลผลลำดับคำทั้งทิศทางไปข้างหน้า (Forward) และย้อนกลับ (Backward) ทำให้เข้าใจความสัมพันธ์ของคำและบริบทประโยคได้อย่างครอบคลุม

ค่า **Hyperparameters** จริงจาก **Notebook**:

- **Word2Vec:** sentences=BIMD_train_tokens, vector_size=100, window=5, min_count=2, sg=1 (Skip-gram), epochs=10, seed=42 (ขนาด Vocabulary = 9,200 คำ + PAD + UNK = 9,202 โทเค็น)
- **Sequence Length:** MAX_LEN = 100 (Mean token length = 16.59, 99th percentile = 89 คำ) โดยทำการ Padding แบบ Post-padding
- **BiLSTM Layer:** LSTM Units = 64 หน่วย (ผลลัพธ์แบบต่อกัน 2 ทิศทาง = 128 มิติ)
- **Regularization:** Dropout Layer 1 = 0.30, Dropout Layer 2 = 0.20
- **Dense Hidden Layer:** 32 หน่วย พร้อมฟังก์ชันกระตุ้น (Activation) แบบ ReLU
- **Output Layer:** 1 หน่วย พร้อมฟังก์ชันกระตุ้นแบบ Sigmoid สำหรับการจำแนกแบบ Binary
- **Training Settings:** Optimizer = Adam (Learning Rate = 0.001), Loss = binary_crossentropy, Batch Size = 64, Epochs = 15
- **Callbacks:** EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True) และ ReduceLROnPlateau(monitor='val_loss', patience=2, factor=0.5, min_lr=1e-6) (หยุดที่ Epoch ที่ 5 และคืนค่าน้ำหนักที่ดีที่สุดจาก Epoch ที่ 2)

Thai Text
↓
Thai Tokenization (PyThaiNLP newmm)
↓
Word2Vec Representation (Skip-gram, dim=100)
↓
Vocabulary Mapping (Vocab Size = 9,202 โทเค็น)
↓
Padding (MAX_LEN = 100, Post-padding)
↓
Embedding Layer (Input: [None, 100], Output: [None, 100, 100], Trainable=True)
↓
Bidirectional LSTM Layer (64 units × 2 = 128 dimensions)
↓
Dropout (Rate = 0.30)
↓
Dense Hidden Layer (32 units, Activation = ReLU)
↓
Dropout (Rate = 0.20)
↓
Dense Output Layer (1 unit, Activation = Sigmoid)
↓
Class Prediction (0: ปกติ / 1: ซึมเศร้า)

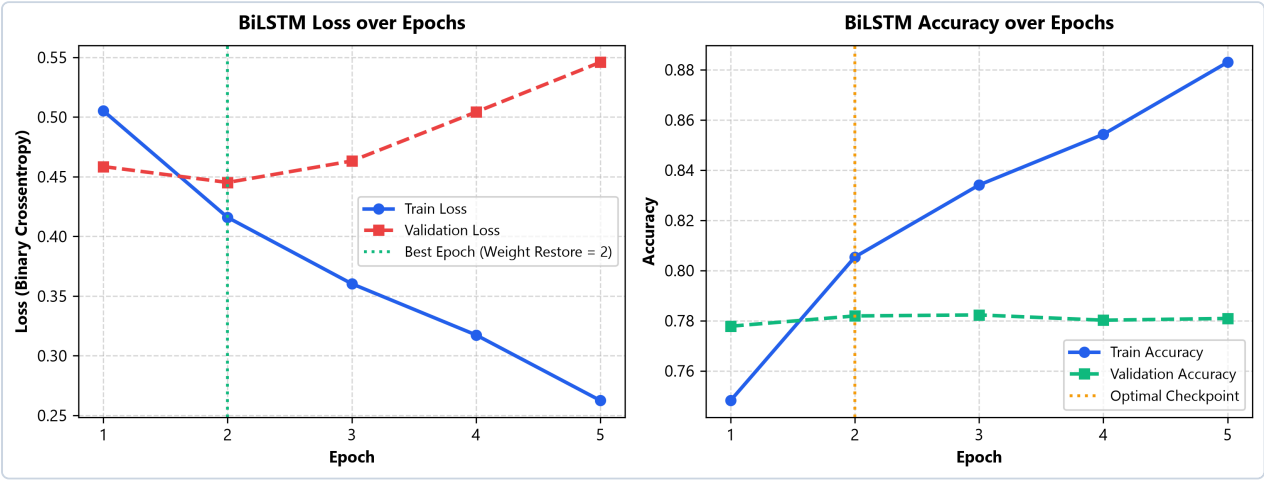
BiLSTM Neural Network Architecture Pipeline (Group 10)



รูปที่ 1: สถาปัตยกรรมโครงข่ายประสาทเทียม BiLSTM ร่วมกับ Word2Vec Embedding

ผลการทดลองจริงบนชุดทดสอบ (Test Set: 4,902 ตัวอย่าง):

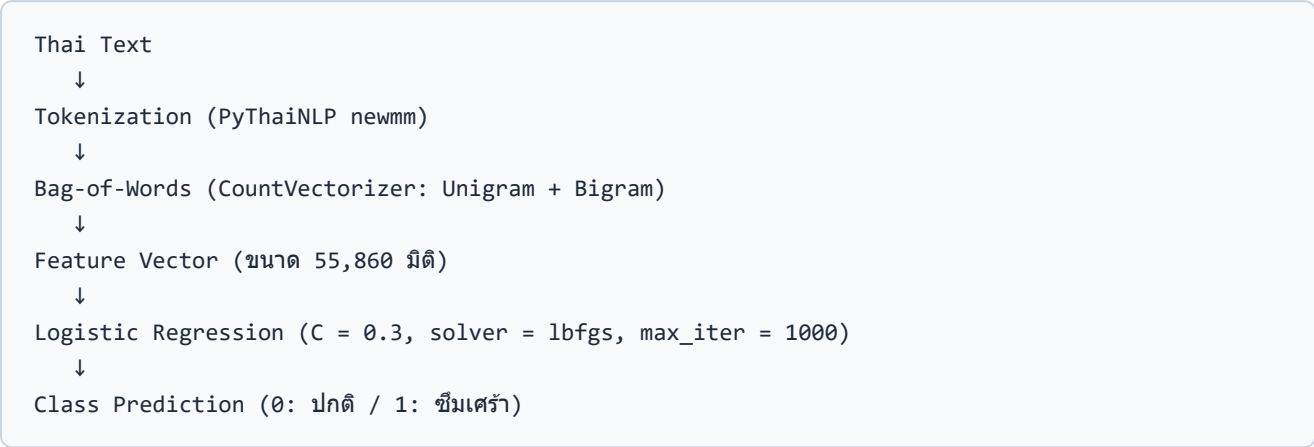
Metric	ผลการทดลองจริง (Result)
Accuracy	0.7840 (78.40%)
Precision (Class 1)	0.7505 (75.05%)
Recall (Class 1)	0.8585 (85.85%) — สูงสุดในทุกโมเดล
F1-score (Class 1)	0.8009 (80.09%) — สูงสุดในทุกโมเดล
ROC-AUC	0.8715 (87.15%)



รูปที่ 2: กราฟแสดงผลการเรียนรู้ Loss และ Accuracy ตลอด Epochs ของโมเดล BiLSTM

7. Model 2: Bag-of-Words (BoW) + Logistic Regression

แนวคิดและหลักการ: เทคนิคพื้นฐานด้าน NLP โดยการแปลงข้อความเป็นเวกเตอร์ความถี่ของการปรากฏของคำ (Word Count Representation) ผ่าน CountVectorizer โดยไม่คำนึงถึงลำดับของคำ จากนั้นใช้แบบจำลองการจำแนกเชิงเส้น **Logistic Regression** ร่วมกับการควบคุมความซับซ้อนด้วย L2 Regularization



การปรับจูน Hyperparameter (C Tuning บน Validation Set):

ค่า C ที่ทดสอบ	Validation Accuracy	Validation Precision	Validation Recall	Validation F1-score	Validation ROC-AUC
0.001	0.6015	0.5607	0.8412	0.6729	0.7226
0.010	0.7248	0.6976	0.7811	0.7371	0.8142
0.100	0.7745	0.7516	0.8247	0.7865	0.8624
0.300 (Best C)	0.7770	0.7577	0.8267	0.7906	0.8643
0.500	0.7760	0.7588	0.8222	0.7892	0.8624
1.000	0.7735	0.7589	0.8142	0.7854	0.8581
3.000	0.7690	0.7581	0.8025	0.7797	0.8491
5.000	0.7668	0.7580	0.7977	0.7773	0.8441
10.000	0.7635	0.7562	0.7912	0.7733	0.8364

ผลการทดลองจริงบนชุดทดสอบ (Test Set: 4,902 ตัวอย่าง) ที่ค่า C = 0.3:

Metric	ผลการทดลองจริง (Result)
Accuracy	0.7783 (77.83%)
Precision (Class 1)	0.7574 (75.74%)
Recall (Class 1)	0.8267 (82.67%)
F1-score (Class 1)	0.7905 (79.05%)
ROC-AUC	0.8660 (86.60%)

8. Model 3: TF-IDF + Logistic Regression

แนวคิดและหลักการ: การแปลงข้อความเป็นเวกเตอร์โดยใช้น้ำหนัก **Term Frequency-Inverse Document Frequency (TF-IDF)** ร่วมกับ `sublinear_tf=True` ซึ่งช่วยลดน้ำหนักของคำที่ปรากฏบ่อยเกินไปในทุกเอกสาร และเน้นคำเฉพาะเจาะจงที่มีความสำคัญต่อการจำแนกคลาส

```
Thai Text
↓
Tokenization (PyThaiNLP newmm)
↓
TF-IDF Vectorizer (ngram_range=(1,2), min_df=2, sublinear_tf=True)
↓
Feature Vector (ขนาด 55,860 มิติ)
↓
Logistic Regression (C = 3.0, solver = lbfgs, max_iter = 1000)
↓
Class Prediction (0: ปกติ / 1: ซึมเศร้า)
```

การปรับจูน Hyperparameter (C Tuning บน Validation Set):

ค่า C ที่ทดสอบ	Validation Accuracy	Validation Precision	Validation Recall	Validation F1-score	Validation ROC-AUC
0.001	0.5892	0.5524	0.9960	0.7107	0.6974
0.010	0.7180	0.6698	0.8783	0.7601	0.8164
0.100	0.7725	0.7441	0.8388	0.7885	0.8601
0.300	0.7790	0.7618	0.8186	0.7892	0.8671
0.500	0.7802	0.7686	0.8090	0.7883	0.8679
1.000	0.7811	0.7788	0.7944	0.7865	0.8682
3.000 (Best C)	0.7813	0.7844	0.7848	0.7846	0.8673
5.000	0.7805	0.7854	0.7815	0.7834	0.8663
10.000	0.7796	0.7866	0.7775	0.7820	0.8643

ผลการทดลองจริงบนชุดทดสอบ (Test Set: 4,902 ตัวอย่าง) ที่ค่า C = 3.0:

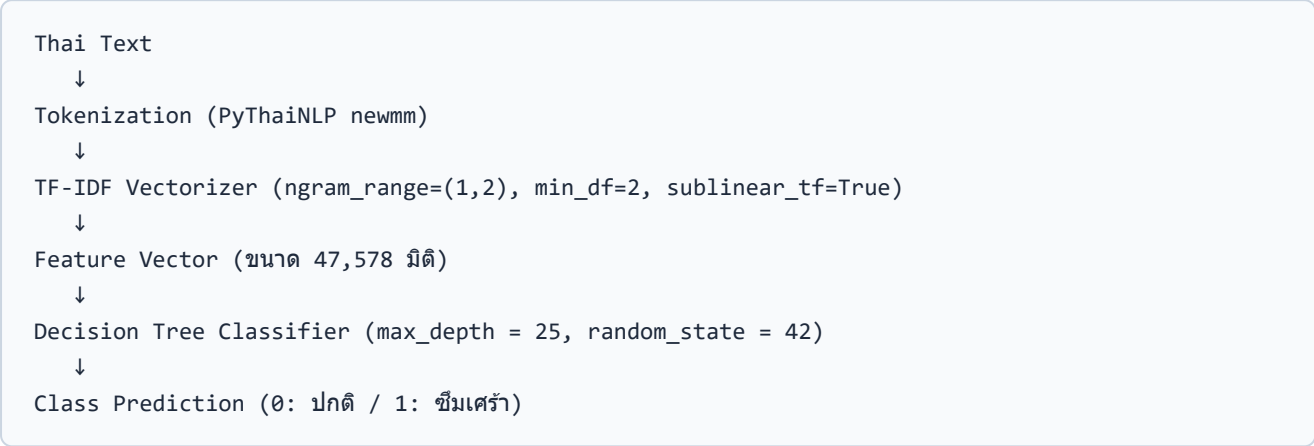
Metric	ผลการทดลองจริง (Result)
Accuracy	0.7815 (78.15%)
Precision (Class 1)	0.7831 (78.31%) — สูงสุดในทุกโมเดล
Recall (Class 1)	0.7860 (78.60%)
F1-score (Class 1)	0.7846 (78.46%)
ROC-AUC	0.8676 (86.76%)

คำสำคัญที่มีค่าน้ำหนักสัมประสิทธิ์สูงสุดในโมเดล (Top Words):

- Class 1 (ซึมเศร้า): ยา, หมอ, เรา, ดรีม, คุณหมอ, โรค, อาการ, ซึมเศร้า, ค่ะ, ป่วย, ชื่น, ฆ่าตัวตาย, การรักษา, ทำร้าย, จิตแพทย์
- Class 0 (ปกติ): เธอ, ความรัก, เต็บโต, พี่, เขา, กบ, ดอกไม้, นั้น, แสงค์, รัก, ของ, นาย, ยาย, ครู, เมียง

9. Model 4: TF-IDF + Decision Tree

แนวคิดและหลักการ: แบบจำลองแบบต้นไม้การตัดสินใจ (Decision Tree) ซึ่งสร้างกฎการแบ่งข้อมูลแบบไม่เป็นเส้นตรง (Non-linear Rules) โดยเนื่องจาก Decision Tree ไม่สามารถรับข้อความดิบได้ จึงต้องแปลงข้อความเป็นเวกเตอร์ TF-IDF (จำนวนฟีเจอร์ 47,578 มิติ) ก่อนนำเข้าสู่กระบวนการตัดสินใจ

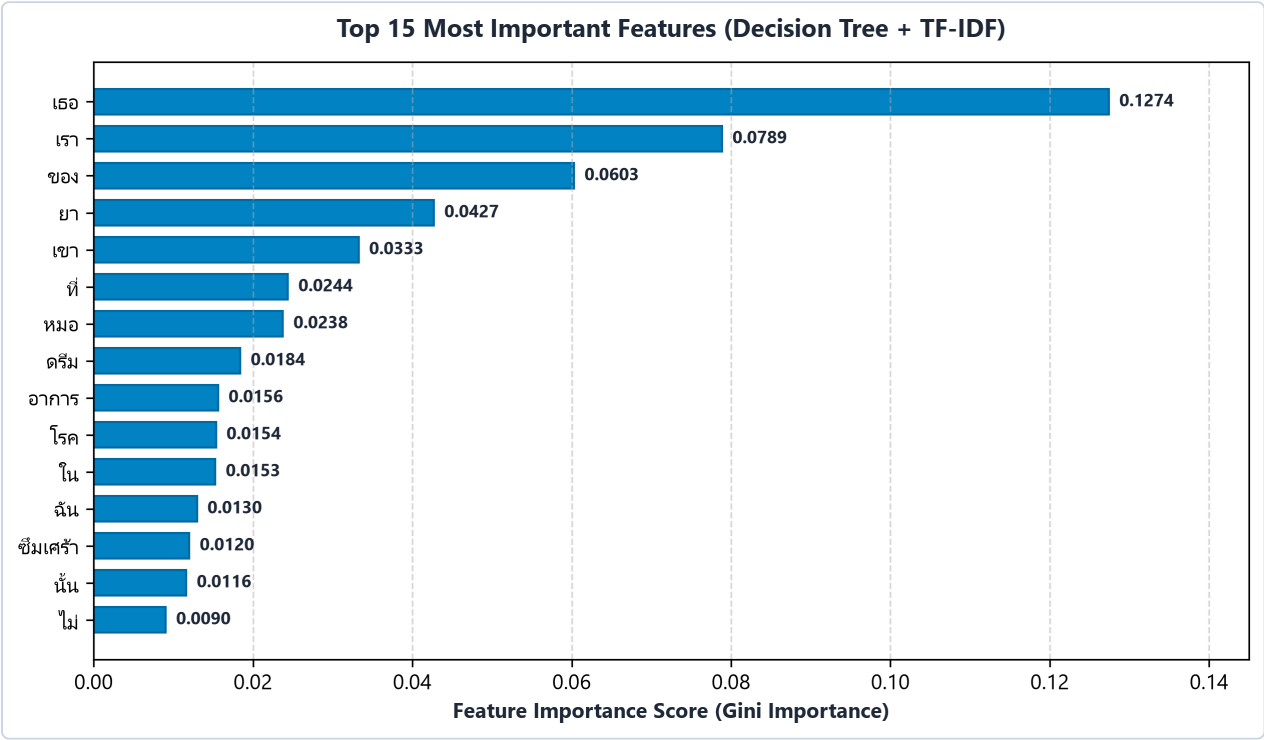


การปรับจูน Hyperparameter (max_depth Tuning บน Validation Set):

max_depth ที่ทดสอบ	Validation Accuracy	Validation Precision	Validation Recall	Validation F1-score	Validation ROC-AUC
5	0.6293	0.5857	0.9141	0.7140	0.6869
10	0.6621	0.6393	0.7630	0.6957	0.7190
15	0.6678	0.6504	0.7432	0.6938	0.7206
20	0.6744	0.6517	0.7662	0.7043	0.7153
25 (Best max_depth)	0.6766	0.6576	0.7533	0.7022	0.7056
30	0.6725	0.6477	0.7743	0.7053	0.6956
50	0.6707	0.6468	0.7699	0.7030	0.6815
None (ไม่จำกัดความลึก)	0.6609	0.6579	0.6840	0.6707	0.6607

ผลการทดลองจริงบนชุดทดสอบ (Test Set: 4,902 ตัวอย่าง) ที่ค่า max_depth = 25:

Metric	ผลการทดลองจริง (Result)
Accuracy	0.6681 (66.81%)
Precision (Class 1)	0.6482 (64.82%)
Recall (Class 1)	0.7529 (75.29%)
F1-score (Class 1)	0.6966 (69.66%)
ROC-AUC	0.7124 (71.24%)



รูปที่ 3: แผนภูมิแสดง Feature Importance 15 อันดับแรกของโมเดล Decision Tree

10. Model 5: BERT (Transformer-based Architecture)

แนวคิดและหลักการ: BERT (Bidirectional Encoder Representations from Transformers) เป็นโมเดลภาษาขนาดใหญ่ที่ใช้สถาปัตยกรรม Transformer Encoder ซึ่งสามารถเรียนรู้บริบทประโยคแบบรอบทิศทางในระดับลึก (Self-Attention Mechanism) เพื่อนำมา Fine-tune สำหรับงาน Binary Text Classification



สถานะและผลการทดลอง:

ส่วนของ BERT จะดำเนินการเพิ่มเติมและเติมผลการทดลองหลังจากพัฒนาและฝึกโมเดลเสร็จสิ้น

Metric	ผลการทดลอง (Result)
Accuracy	Pending
Precision	Pending
Recall	Pending
F1-score	Pending

11. ตัวชี้วัดประสิทธิภาพ (Evaluation Metrics)

การประเมินผลการจำแนกข้อความใช้ตัวชี้วัดมาตรฐาน 4 ตัวหลัก โดยอ้างอิงจากตารางความสับสน (Confusion Matrix) ซึ่งมีองค์ประกอบดังนี้:

- True Positive (TP):** จำนวนข้อความที่บ่งชี้ภาวะซึมเศร้า (Class 1) และโมเดลทำนายได้ถูกต้องเป็น Class 1
- True Negative (TN):** จำนวนข้อความปกติ (Class 0) และโมเดลทำนายได้ถูกต้องเป็น Class 0
- False Positive (FP):** จำนวนข้อความปกติ (Class 0) แต่โมเดลทำนายผิดพลาดว่าเป็น Class 1 (เกิด False Alarm)
- False Negative (FN):** จำนวนข้อความที่บ่งชี้ภาวะซึมเศร้า (Class 1) แต่โมเดลทำนายผิดพลาดว่าเป็น Class 0 (พลาดเคสเสี่ยง)

1. Accuracy (ความถูกต้องรวม):

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

สัดส่วนของข้อความทั้งหมดที่โมเดลทำนายถูกต้องทั้ง Class 0 และ Class 1

2. Precision (ความแม่นยำในการทำนายเชิงบวก):

$$\text{Precision} = TP / (TP + FP)$$

สัดส่วนความถูกต้องเมื่อโมเดลทำนายว่าเป็นภาวะซึมเศร้า (Class 1)

3. Recall (ความสามารถในการตรวจจับ / Sensitivity):

$$\text{Recall} = TP / (TP + FN)$$

สัดส่วนของข้อความซึมเศร้าจริงทั้งหมดที่โมเดลสามารถตรวจจับได้

4. F1-score (ค่าเฉลี่ยฮาร์โมนิกของ Precision และ Recall):

$$\text{F1-score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

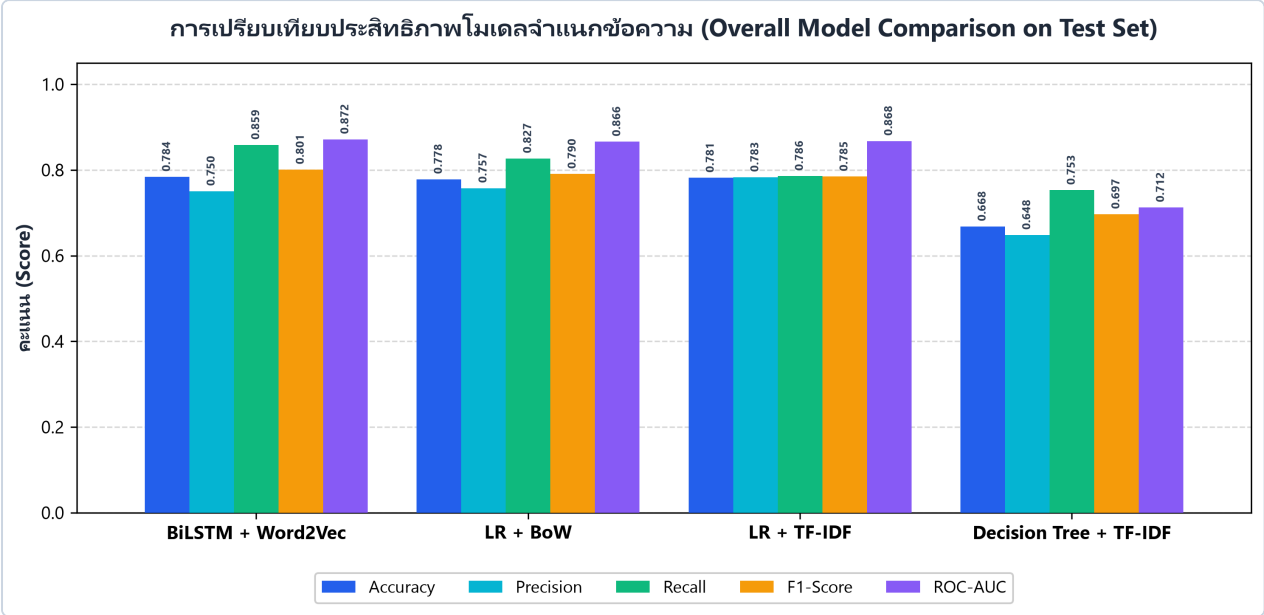
ตัวชี้วัดความสมดุลระหว่างความแม่นยำและการตรวจจับ

ความสำคัญของ Recall สำหรับงานคัดกรองภาวะซึมเศร้า: ในบริบทของการคัดกรองสุขภาพจิต ค่า **Recall ของ Class 1** มีความสำคัญอย่างยิ่ง เนื่องจากเป้าหมายหลักคือการลด False Negative (การปล่อยให้ผู้ที่มีภาวะเสี่ยงซึมเศร้าหลุดรอดไปโดยไม่ได้รับการช่วยเหลือ) ให้เหลือน้อยที่สุด

12. ผลการทดลองรวม (Overall Experimental Results)

ตารางเปรียบเทียบผลลัพธ์จริงที่ได้จากการประเมินบนชุดข้อมูลทดสอบ (Test Set: 4,902 ตัวอย่าง) ของทุกโมเดล:

โมเดล / สถาปัตยกรรม (Model)	Accuracy	Precision (Class 1)	Recall (Class 1)	F1-score (Class 1)	ROC-AUC
1. Word2Vec + BiLSTM	0.7840 (78.40%)	0.7505 (75.05%)	0.8585 (85.85%)	0.8009 (80.09%)	0.8715 (87.15%)
2. BoW + Logistic Regression	0.7783 (77.83%)	0.7574 (75.74%)	0.8267 (82.67%)	0.7905 (79.05%)	0.8660 (86.60%)
3. TF-IDF + Logistic Regression	0.7815 (78.15%)	0.7831 (78.31%)	0.7860 (78.60%)	0.7846 (78.46%)	0.8676 (86.76%)
4. TF-IDF + Decision Tree	0.6681 (66.81%)	0.6482 (64.82%)	0.7529 (75.29%)	0.6966 (69.66%)	0.7124 (71.24%)
5. BERT (Transformer)	Pending	Pending	Pending	Pending	Pending



รูปที่ 4: แผนภูมิแท่งเปรียบเทียบผลการทดลองบน Test Set ทุกตัวชี้วัด

13. การวิเคราะห์เปรียบเทียบโมเดล (Model Comparison)

การวิเคราะห์เชิงเปรียบเทียบความแตกต่างระหว่างคุณลักษณะของแต่ละโมเดลทั้งในแง่ของ Feature Extraction, ความเข้าใจบริบท, การเรียงลำดับคำ และความซับซ้อน:

Model	Feature Type	Context Awareness	Sequence Modeling	Complexity	จุดเด่น / ข้อจำกัด
BoW + LR	Word Count (BoW)	ต่ำ	ไม่รองรับ	ต่ำ	เทรนเร็วมาก แต่ไม่คำนึงถึงบริบทและลำดับคำ
TF-IDF + LR	Weighted TF-IDF	ต่ำ	ไม่รองรับ	ต่ำ	ตัดคำรบกวนได้ดี Precision สูง แต่ไม่เข้าใจไวยากรณ์ประโยค
TF-IDF + DT	Weighted TF-IDF	ต่ำ	ไม่รองรับ	ต่ำ-กลาง	อธิบายการตัดสินใจได้ง่าย แต่ประสิทธิภาพต่ำบน Sparse Text
Word2Vec + BiLSTM	Dense Embedding	สูง	รองรับ (2 ทิศทาง)	สูง	จับความสัมพันธ์ของคำและความหมายแฝงได้ดี Recall สูงสุด
BERT	Contextual Token	สูงมาก	รองรับ (Self-Attention)	สูงมาก	เข้าใจภาษาระดับ State-of-the-Art (อยู่ระหว่างดำเนินการ)

14. การเลือกโมเดลที่ดีที่สุด (Best Model Selection)

จากการวิเคราะห์ผลการทดลองจริงที่มีอยู่ใน Notebook สามารถสรุปโมเดลที่ดีที่สุดในการทดลองปัจจุบันได้ดังนี้:

จากผลการทดลองพบว่าโมเดล **Word2Vec Embedding + BiLSTM** ให้ผลลัพธ์ที่ดีที่สุดในชุดการทดลองที่มีอยู่ โดยมี **Accuracy = 0.7840 (78.40%)**, **Precision = 0.7505 (75.05%)**, **Recall = 0.8585 (85.85%)** และ **F1-score = 0.8009 (80.09%)** รวมถึงมีค่า **ROC-AUC** สูงสุดที่ **0.8715**

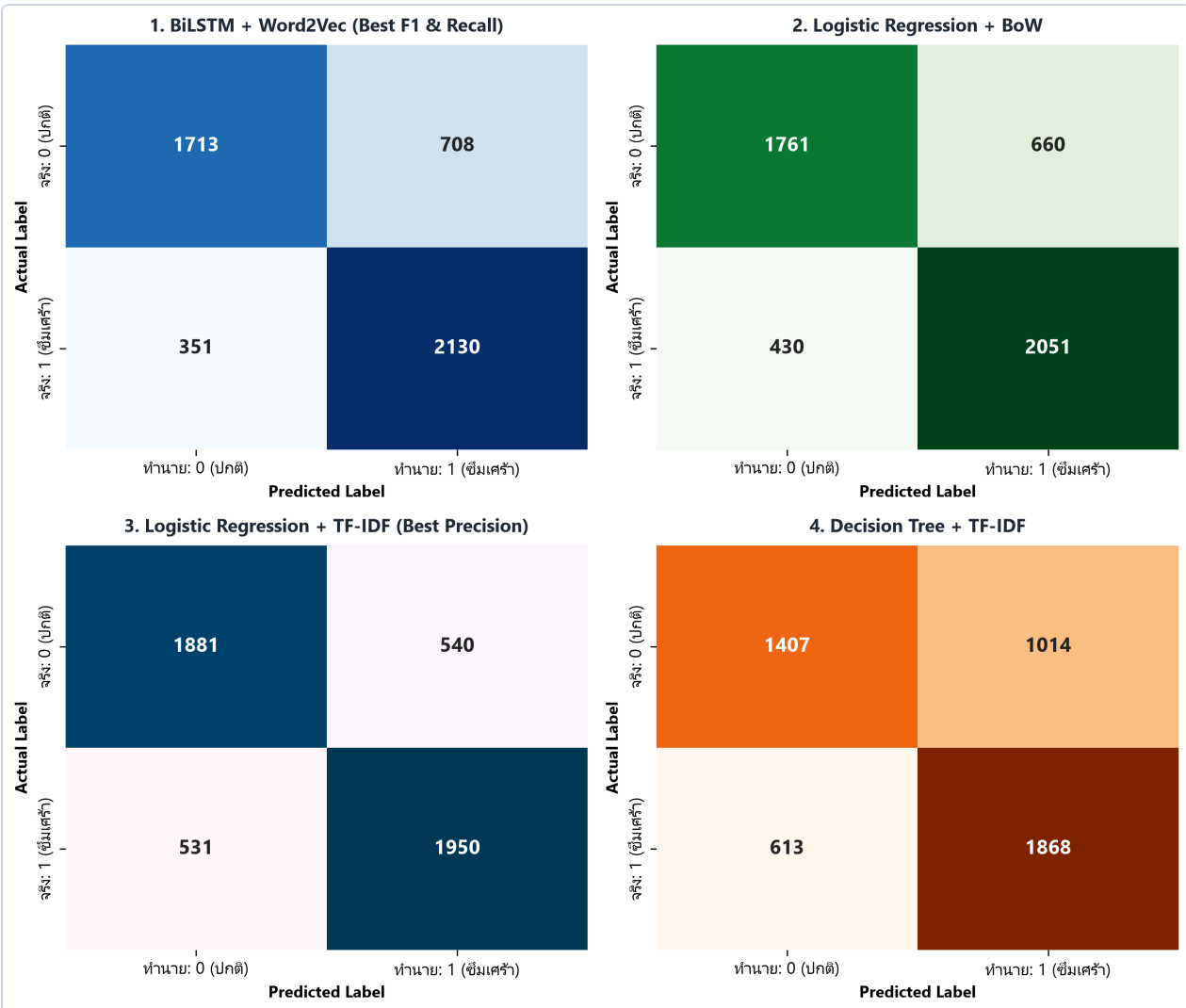
อย่างไรก็ตาม เมื่อพิจารณาตามข้อกำหนดของโครงการที่ต้องเปรียบเทียบกับโมเดลกลุ่ม Transformer:

ยังไม่สามารถสรุป **Best Model** ขั้นสุดท้ายได้ เนื่องจากการทดลอง **BERT** ยังไม่เสร็จสมบูรณ์

15. เมทริกซ์ความสับสน (Confusion Matrix Analysis)

เมทริกซ์ความสับสน (Confusion Matrix) แสดงโครงสร้างการจำแนกตัวอย่างจริงเทียบกับค่าทำนาย:

		Predicted Class	
		0 (ปกติ)	1 (ซึมเศร้า)
Actual Class	0	TN	FP
	1	FN	TP



รูปที่ 5: เมทริกซ์ความสับสนจริงของทั้ง 4 โมเดลบน Test Set (จำนวนตัวอย่างรวม 4,902 ตัวอย่าง)

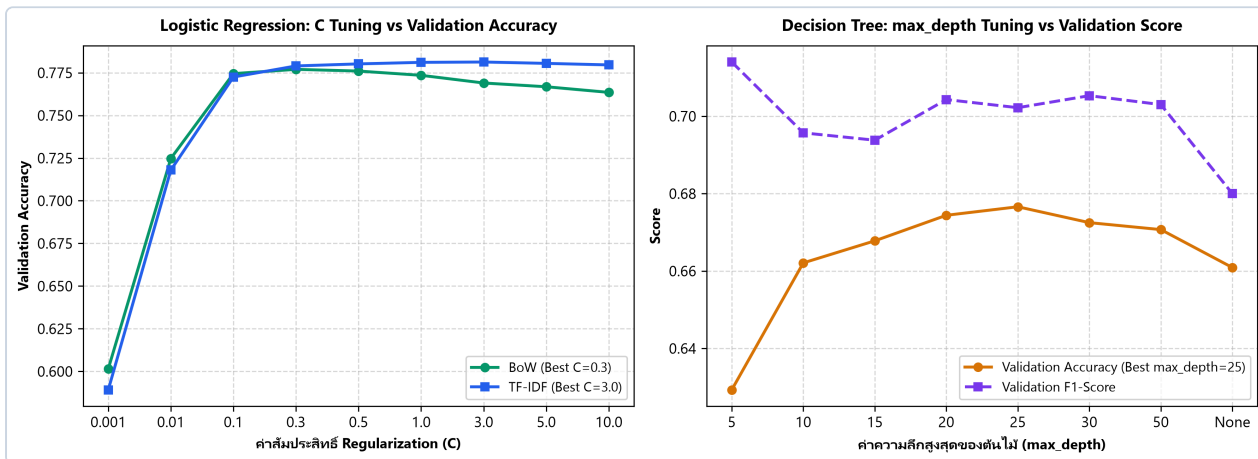
การวิเคราะห์ False Positive และ False Negative:

- **BiLSTM + Word2Vec:** ให้ค่า FN ต่ำที่สุดเพียง 351 ตัวอย่าง (ตรวจจับเคสซึมเศร้าสำเร็จ 2,130 จาก 2,481 ตัวอย่าง) มีความปลอดภัยสูงในการนำไปเป็นเครื่องมือคัดกรองเบื้องต้น
- **Logistic Regression + TF-IDF:** ให้ค่า FP ต่ำที่สุดเพียง 540 ตัวอย่าง (ทำนาย Class 0 ถูกต้อง 1,881 ตัวอย่าง) ลดปัญหาการแจ้งเตือนผิดพลาด (False Alarm)
- **Decision Tree + TF-IDF:** มีความผิดพลาดทั้ง FP (1,014 ตัวอย่าง) และ FN (613 ตัวอย่าง) ในระดับสูงกว่าโมเดลอื่น

16. การปรับจูนพารามิเตอร์ (Hyperparameter Tuning Summary)

ตารางสรุปค่า Hyperparameters ที่ทำการทดลองปรับจูนจริงใน Notebook เพื่อค้นหาค่าที่เหมาะสมที่สุด:

ชื่อโมเดล (Model)	พารามิเตอร์ที่ทำการ จูน	ช่วงค่าที่ทดสอบ (Values Tested)	ค่าที่ดีที่สุด (Best Value)	คะแนนอ้างอิงบน Validation Set
BoW + Logistic Regression	Regularization Strength (C)	[0.001, 0.01, 0.1, 0.3, 0.5, 1.0, 3.0, 5.0, 10.0]	C = 0.3	Val Accuracy: 0.7770 (F1: 0.7906)
TF-IDF + Logistic Regression	Regularization Strength (C)	[0.001, 0.01, 0.1, 0.3, 0.5, 1.0, 3.0, 5.0, 10.0]	C = 3.0	Val Accuracy: 0.7813 (F1: 0.7846)
TF-IDF + Decision Tree	Maximum Tree Depth (max_depth)	[5, 10, 15, 20, 25, 30, 50, None]	max_depth = 25	Val Accuracy: 0.6766 (F1: 0.7022)
Word2Vec + BiLSTM	LSTM Units & Architecture	LSTM units: 64, Dense: 32, Dropout: 0.3/0.2	Fixed 64 units	Val Accuracy: 0.7819 (Epoch 2 Checkpoint)



รูปที่ 6: กราฟแสดงความสัมพันธ์ระหว่างค่า Hyperparameter และคะแนนการประเมินบน Validation Set

17. ข้อจำกัดของงาน (Limitations)

จากการวิเคราะห์เชิงลึกของชุดข้อมูลและกระบวนการทดลอง พบข้อจำกัดสำคัญดังนี้:

- ความซับซ้อนของภาษาไทยบนสื่อออนไลน์:** ข้อความภาษาไทยบน Social Media มีการใช้คำแสลง คำย่อ การเขียนสะกดคำแบบไม่เป็นทางการ และการละเว้นเครื่องหมายวรรคตอน ซึ่งอาจส่งผลกระทบต่อความแม่นยำในการตัดคำ (Word Tokenization Errors)
- ข้อความสั้นและบริบทจำกัด:** ข้อความบางส่วนในชุดข้อมูลมีความยาวสั้นมาก (เช่น มีความยาวเพียง 3-10 ตัวอักษร) ทำให้ไม่มีข้อมูลบริบทเพียงพอสำหรับแบบจำลองในการตัดสินใจที่แท้จริง
- ข้อจำกัดด้านบริบทของโมเดล BoW และ TF-IDF:** โมเดลกลุ่ม Bag-of-Words และ TF-IDF ไม่สามารถเข้าใจลำดับของคำ (Word Order) และความหมายที่เปลี่ยนไปตามการใช้คำปฏิเสธ (Negation Handling) เช่น ความแตกต่างระหว่างคำว่า "อยากตาย" กับ "ไม่อยากตาย"

4. **ทรัพยากรการประมวลผลของโมเดล Deep Learning:** โครงข่าย BiLSTM ใช้เวลาและทรัพยากรในการฝึกมากกว่า Linear Models อย่างมีนัยสำคัญ
5. **การทดลองของโมเดล BERT ยังไม่สมบูรณ์:** แบบจำลอง Transformer-based Language Model (BERT) ยังอยู่ระหว่างขั้นตอนการพัฒนา ทำให้ยังไม่สามารถเปรียบเทียบผลลัพธ์กับโมเดลยุคปัจจุบันได้อย่างครบถ้วน

18. บทสรุปโครงการ (Conclusion)

กลุ่มที่ 10 (Group 10) ได้ดำเนินการศึกษา ออกแบบ และเปรียบเทียบแบบจำลองการจำแนกข้อความภาษาไทยเพื่อระบุภาวะซึมเศร้า ครอบคลุมตั้งแต่แบบจำลอง Traditional Machine Learning (Logistic Regression, Decision Tree) ไปจนถึง Deep Sequential Model (BiLSTM ร่วมกับ Pre-trained Word2Vec) บนชุดข้อมูลขนาด 32,676 ข้อความ

จากผลการทดลองจริงบนชุดทดสอบ (Test Set: 4,902 ตัวอย่าง) สามารถสรุปสาระสำคัญได้ดังนี้:

- **BiLSTM + Word2Vec** เป็นโมเดลที่มีประสิทธิภาพสูงสุดในชุดการทดลองปัจจุบัน โดยได้ **Accuracy = 78.40%, Recall = 85.85%, F1-score = 80.09%** และ **ROC-AUC = 87.15%** เนื่องจากสามารถจับลำดับคำและความสัมพันธ์เชิงบริบทสองทิศทางได้ดี
- **Logistic Regression + TF-IDF (C = 3.0)** เป็นโมเดลเชิงเส้นที่มีความสมดุลสูงมาก โดยให้ค่า **Precision สูงที่สุดที่ 78.31%** และ **Accuracy = 78.15%** ด้วยข้อได้เปรียบด้านความเร็วในการประมวลผล
- **Logistic Regression + BoW (C = 0.3)** ให้ผลลัพธ์ในเกณฑ์ดี (Accuracy = 77.83%, Recall = 82.67%)
- **Decision Tree + TF-IDF (max_depth = 25)** มีประสิทธิภาพต่ำที่สุดในกลุ่มการทดลอง (Accuracy = 66.81%, F1 = 69.66%) เนื่องจากข้อจำกัดในการแบ่งข้อมูลแบบ Axis-aligned บนเวกเตอร์ข้อความที่มีมิติสูงและเบาบาง (High-dimensional Sparse Data)
- **BERT:** อยู่ในสถานะ Pending ซึ่งจะดำเนินการเพิ่มเติมในลำดับถัดไปเพื่อนำมาเปรียบเทียบและสรุป Best Model ขั้นสุดท้ายต่อไป